COP3330C Module 14 Programming Assignment 6

Objective: Implement a simple Java program that implements the Model-View-Controller (MVC) pattern to manage a list of user-entered favorite books. The program accepts input from the user (via System.in), writes and reads the list to/from a file using serialization and deserialization via I/O streams, and displays the list to the console.

Design Notes:

MVC Pattern

The MVC (Model-View-Controller) pattern is known for its ability to separate different aspects of an application, making it flexible and modular. In the example of a Java program managing a list of favorite books through a command-line interface, the MVC pattern offers the following flexibility to replace or modify components independently:

> **Model:** The Model represents the core data and the operations that can be performed on it. In this case, it manages the list of books, stores data about each book, and provides methods for adding, removing, or retrieving books. Since the Model contains only the business logic and data representation, it can remain unchanged if you decide to change how users interact with the application. For instance, whether users interact via a command-line interface, a web interface, or a mobile app, the Model's functionality stays the same.
>
> **View:** The View handles user interaction by displaying data and accepting input. In this example, the View is a simple command-line interface that outputs book details to the console and accepts user input from System.in.Because the View is completely separated from the Model, it can easily be replaced with another type of user interface. For instance, instead of using the console, the View can be replaced by:
>
> - A GUI View: Using a framework like JavaFX to create a desktop graphical interface.
> - A Web View: Using a web application front end, where the input/output is handled via a web browser
>
> **Controller:** The Controller manages the communication between the View and Model. It takes input from the View, processes it, and updates the Model accordingly. It also retrieves data from the Model to pass back to the View. Because the Controller contains the logic that links the View and Model, it can be updated to accommodate different types of Views. For example, if a new web-based View is implemented, the Controller may need only minor changes to adapt to new methods of receiving input or delivering data.

The MVC pattern **loosely couples** these three components. Changes in one part of the system do not directly affect the others. In this way it promotes the separation of concerns, which ultimately

leads to better maintainability, scalability, and the ability to extend and modify different parts of the application without affecting others

Serialization and Deserialization

Serialization and deserialization are processes for storing and retrieving objects persistently. Serialization involves converting the in-memory representation of objects into a format suitable for storage, such as writing a bytestream to a file, while deserialization involves reconstructing the objects back into memory from the stored format. In this exercise, serialization is used to write data to a file, ensuring that the current state is captured and can be retrieved later. Conversely, deserialization is used to read the stored data back, restoring it to its original structure in memory. This provides an efficient way to persist the book data, allowing the program to save user input, maintain it across sessions, and easily reload it when needed. By using I/O streams for serialization and deserialization, this abstracts the complexities of managing raw data and provides a seamless mechanism for working with complex data structures.

How It Works:

1. **Run the Program:**
    - The user is presented with a menu to manage their book list.
2. **Add Books:**
    - The user can add book titles to the list.
3. **Save to File:**
    - Books are saved to a file (books.dat) using serialization.
4. **Load from File:**
    - Books are loaded back into the program.
5. **View Books:**
    - Displays the list of books stored in memory.
6. **Exit:**
    - The program ends when the user chooses the exit option.

The GitHub Classroom repo provides starter code which implements most of the application. Your job is to complete the two TODO: items in the BookController to create the required I/O Stream objects in order for the serialization and deserialization to work correctly.

Submit your solution to the repo (retain the original project structure) and a screen snip showing your program execution.

Sample Run (user input shown in red font)

```
Book Manager:
1. Add a book
2. View all books
```

```
3. Save books to file
4. Load books from file
5. Exit
Enter your choice: 1
Enter book title: The Catcher in the Rye
Book added: The Catcher in the Rye

Book Manager:
1. Add a book
2. View all books
3. Save books to file
4. Load books from file
5. Exit
Enter your choice: 2

Books:
1. The Catcher in the Rye

Book Manager:
1. Add a book
2. View all books
3. Save books to file
4. Load books from file
5. Exit
Enter your choice: 3
Books saved to file: books.dat

Book Manager:
1. Add a book
2. View all books
3. Save books to file
4. Load books from file
5. Exit
Enter your choice: 5
Exiting program...
```